

Project Code:

```
/*
 * AgroSmart - ESP32 Node (LoRa E5-HF)
 * Final Clean Code
 */

#include <Arduino.h>

#include <DHT.h>

#include <Wire.h>

#include <RTCLib.h>

// — PINS —————

#define DHTPIN    4

#define DHTTYPE   DHT11

#define MOISTURE_PIN 34

#define BUZZER_PIN 14

#define RELAY_PIN 13

#define WATER_VCC_PIN 25

#define WATER_SIG_PIN 35

// — THRESHOLDS —————

#define MOISTURE_LOW    40 // ← changed 30→50

#define MOISTURE_CRITICAL 35 // ← changed 20→35

#define TEMP_CRITICAL   38.0

DHT dht(DHTPIN, DHTTYPE);

RTC_DS3231 rtc;

bool pumpOn = false;
```

```
HardwareSerial LoRaSerial(2);
```

```
// — STRING → HEX —————
```

```
String stringToHex(const String& input) {  
    String hex = "";  
    char buf[3];  
    for (int i = 0; i < (int)input.length(); i++) {  
        sprintf(buf, "%02X", (uint8_t)input[i]);  
        hex += buf;  
    }  
    return hex;  
}
```

```
// — BUZZER —————
```

```
void buzz(int times, int onMs = 200, int offMs = 150) {  
    for (int i = 0; i < times; i++) {  
        digitalWrite(BUZZER_PIN, HIGH); delay(onMs);  
        digitalWrite(BUZZER_PIN, LOW);  
        if (i < times - 1) delay(offMs);  
    }  
}
```

```
// — PUMP —————
```

```
void pumpON() {  
    digitalWrite(RELAY_PIN, HIGH);  
    if (!pumpOn) {  
        pumpOn = true;
```

```
    buzz(1, 200);  
    Serial.println("[PUMP] ON");  
}  
}
```

```
void pumpOFF() {  
    digitalWrite(RELAY_PIN, LOW);  
    if (pumpOn) {  
        pumpOn = false;  
        buzz(2, 100, 100);  
        Serial.println("[PUMP] OFF");  
    }  
}
```

```
// — WATER LEVEL (read only, no block) —  
int readWaterLevel() {  
    digitalWrite(WATER_VCC_PIN, HIGH);  
    delay(100);  
    int raw = analogRead(WATER_SIG_PIN);  
    digitalWrite(WATER_VCC_PIN, LOW);  
    return constrain(map(raw, 0, 4095, 0, 100), 0, 100);  
}
```

```
// — LORA AT COMMAND —————  
String loraSend(const String& cmd, int waitMs = 1500) {  
    LoRaSerial.println(cmd);  
    String resp = "";  
    unsigned long start = millis();
```

```

while (millis() - start < (unsigned long)waitMs) {
    while (LoRaSerial.available()) {
        resp += (char)LoRaSerial.read();
    }
}

resp.trim();

Serial.println("[LORA CMD] " + cmd);

Serial.println("[LORA RSP] " + resp);

return resp;
}

// — LORA INIT —————

bool loraInit() {
    LoRaSerial.begin(9600, SERIAL_8N1, 16, 17);
    delay(1500);
    loraSend("AT", 1000);
    loraSend("AT+MODE=TEST", 2000);
    loraSend("AT+RESET", 3000);
    loraSend("AT+MODE=TEST", 2000);
    String rsp = loraSend("AT+TEST=RFCFG,868,9,125,8,8,14", 2000);
    if (rsp.indexOf("ERROR") >= 0) {
        Serial.println("[LORA] RFCFG FAILED");
        buzz(3, 300, 100);
        return false;
    }
    Serial.println("[LORA] READY");
    buzz(1, 500);
    return true;
}

```

```
}
```

```
// — BUILD JSON —————
```

```
String buildPayload(float temp, float hum, int moisture, int water, bool pump) {
```

```
    DateTime now = rtc.now();
```

```
    char ts[14];
```

```
    sprintf(ts, "%02d%02d%02d%02d%02d%02d",
```

```
            now.year() % 100, now.month(), now.day(),
```

```
            now.hour(), now.minute(), now.second());
```

```
    String json = "{";
```

```
    json += "\"id\": \"N1\",";
```

```
    json += "\"ts\": \"" + String(ts) + "\",";
```

```
    json += "\"temp\": " + String(temp, 1) + ",";
```

```
    json += "\"hum\": " + String(hum, 1) + ",";
```

```
    json += "\"sm\": " + String(moisture) + ",";
```

```
    json += "\"st\": 0,";
```

```
    json += "\"dist\": 0,";
```

```
    json += "\"wl\": " + String(water) + ",";
```

```
    json += "\"rly\": " + String(pump ? 1 : 0) + ",";
```

```
    json += "\"pump\": " + String(pump ? 1 : 0) + ",";
```

```
    json += "\"duty\": " + String(pump ? 100 : 0);
```

```
    json += "}";
```

```
    return json;
```

```
}
```

```
// — SEND VIA LORA —————
```

```
void loraSendData(float temp, float hum, int moisture, int water, bool pump) {
```

```
    String json = buildPayload(temp, hum, moisture, water, pump);
```

```

Serial.print("[LORA] Payload (");
Serial.print(json.length());
Serial.println(" bytes): " + json);
if (json.length() > 115) {
    Serial.println("[LORA ERROR] Payload too large!");
    return;
}

String hexPayload = stringToHex(json);
String cmd = "AT+TEST=TXLRPKT,\"\" + hexPayload + "\"\"";
delay(300);

String resp = loraSend(cmd, 3000);
if (resp.indexOf("ERROR") >= 0) {
    Serial.println("[LORA] Retrying TX...");
    delay(2000);
    resp = loraSend(cmd, 3000);
}

if (resp.indexOf("TXLRPKT") >= 0 || resp.indexOf("TX DONE") >= 0) {
    Serial.println("[LORA SUCCESS]");
} else {
    Serial.println("[LORA ERROR] TX failed: " + resp);
}
}

// — SETUP —————
void setup() {
    Serial.begin(115200);
    delay(2000);

```

```
pinMode(RELAY_PIN, OUTPUT);
```

```
digitalWrite(RELAY_PIN, LOW);
```

```
pumpOn = false;
```

```
pinMode(BUZZER_PIN, OUTPUT);
```

```
digitalWrite(BUZZER_PIN, LOW);
```

```
pinMode(WATER_VCC_PIN, OUTPUT);
```

```
digitalWrite(WATER_VCC_PIN, LOW);
```

```
Wire.begin(21, 22);
```

```
if (!rtc.begin()) {
```

```
    Serial.println("[RTC] FAIL");
```

```
    buzz(3);
```

```
} else {
```

```
    Serial.println("[RTC] OK");
```

```
    if (rtc.lostPower()) rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
```

```
}
```

```
dht.begin();
```

```
delay(2000);
```

```
loraInit();
```

```
digitalWrite(RELAY_PIN, LOW);
```

```
pumpOn = false;
```

```
Serial.println("=== AgroSmart Boot OK ===");
```

```
}
```

```
// — LOOP —————
```

```
void loop() {
```

```
    delay(2000);
```

```
    // — RTC —
```

```
    DateTime now = rtc.now();
```

```
    Serial.printf("[RTC] %04d-%02d-%02d %02d:%02d:%02d\n",
```

```
        now.year(), now.month(), now.day(),
```

```
        now.hour(), now.minute(), now.second());
```

```
    // — DHT11 with retry —
```

```
    float hum = dht.readHumidity();
```

```
    float temp = dht.readTemperature();
```

```
    if (isnan(hum) || isnan(temp)) {
```

```
        delay(500);
```

```
        hum = dht.readHumidity();
```

```
        temp = dht.readTemperature();
```

```
    }
```

```
    if (isnan(hum) || isnan(temp)) {
```

```
        Serial.println("[DHT11] FAIL -- skipping loop");
```

```
        buzz(2, 100, 100);
```

```
        return;
```

```
    }
```

```
    Serial.printf("[DHT11] Temp: %.1f C Hum: %.1f %%\n", temp, hum);
```

```
    if (temp > TEMP_CRITICAL) {
```

```
        Serial.println("[ALERT] High temp!");
```



```

    buzz(3, 300, 100);
}

// — MOISTURE —
int raw = analogRead(MOISTURE_PIN);
if (raw < 10) {
    Serial.println("[MOISTURE] Disconnected -- pump OFF");
    pumpOFF();
    Serial.println("-----");
    return;
}
int pct = constrain(map(raw, 0, 4095, 0, 100), 0, 100);
Serial.printf("[MOISTURE] %d %%\n", pct);

// — WATER LEVEL (log only, no block) —
int waterPct = readWaterLevel();
Serial.printf("[WATER] %d %%\n", waterPct);

// — IRRIGATION LOGIC —————
// Only moisture controls pump now
if (pct < MOISTURE_LOW) {
    pumpON();
    if (pct < MOISTURE_CRITICAL) {
        Serial.println("[ALERT] CRITICAL DRY!");
        buzz(2, 500, 200);
    }
} else {
    pumpOFF();
}

```

```
}
```

```
Serial.printf("[PUMP STATE] %s\n", pumpOn ? "ON" : "OFF");
```

```
loraSendData(temp, hum, pct, waterPct, pumpOn);
```

```
Serial.println("-----");
```

```
}
```